

Hybrid graph structure learning for improving semantic dependency parsing with robust graph neural networks

Irkham Huda^{a,b}, Moh Edi Wibowo^a, Reza Pulungan^a^{*}

^a Department of Computer Science and Electronics, Faculty of Mathematics and Natural Sciences, Universitas Gadjah Mada, Yogyakarta, Indonesia

^b Department of Electrical Engineering and Informatics, School of Vocational Studies, Universitas Gadjah Mada, Yogyakarta, Indonesia

ARTICLE INFO

Communicated by Y. Cheng

Dataset link: <https://simpan.ugm.ac.id/s/cBRvB2mNi1JdsQ1>

Keywords:

Graph structure learning

Graph neural network

Graph isomorphism network

Graph attention network

ABSTRACT

Graph neural networks (GNN) can learn powerful representations of graphs, but their effectiveness is heavily influenced by the quality of the graph structure used. Graph structure learning (GSL) was developed to extract useful graph structures from data and learn better graph representations using GNN. GSL creates an adjacency matrix with two multilayer perceptrons and a biaffine attention network. However, it does not directly incorporate the principles of homophily, sparsity, and degree distribution, which are important in many real-life situations. Differentiable graph structure learning neural networks (DGSLN), which adhere to these basic principles, have been proven to improve GNN performance. This study proposes a GNN-based semantic dependency parsing model that incorporates the GSL adapted from the DGSLN approach to generate an initial graph. For the graph representation learning stage, we use GNN variants, GIN and GATv2, which have shown robust performance in previous research. The proposed model significantly improves previous best models, achieving an average F1 score of 93.77% on in-domain and 92.27% on out-of-domain datasets. However, our model's parsing performance is slower than prior top-performing models. The model utilizing GIN exhibits the highest level of performance and shows exceptional performance in semantic dependency parsing on English datasets.

1. Introduction

Syntactic and semantic parsing is a technique utilized in natural language processing (NLP) to uncover the underlying structural connections between words in sentences, as well as the grammatical principles that dictate sentence construction. Parsing is crucial in diverse NLP applications, including machine translation [1], question answering [2], sentiment analysis [3], and plagiarism detection [4]. In order to accurately represent syntactic and semantic information in sentences, a grammar system is needed. The grammar widely adopted for parsing is dependency grammar, also known as semantic dependency parsing (SDP). SDP constructs a labeled directed acyclic graph (DAG), referred to as the semantic dependency graph (SDG), which visualizes the semantic relations between sentence words [5].

SDP research has grown in recent years. Most SDP models are transition or graph-based. Transition-based dependency parsing builds a valid parse tree (SDG) incrementally from sequences of actions. The parser's action choices determine word relationships. Transition-based models use local and greedy predictions to parse sentences faster,

but they are more prone to errors because they lack global inference [6,7]. Various types of models exist for transition-based dependency parsing. These include statistical models employing various strategies, such as arc-standard and arc-eager [8], beam-search [9], rich global features [10], and dynamic oracle [11]. In addition, there are models based on neural networks that improve the parsing process by utilizing feedforward networks [12] and stacked LSTMs [13,14].

Graph-based dependency parsing views SDP as a graph problem, where words in a sentence are represented as nodes, and the connections between them are represented as edges [5]. Graph-based dependency parsing typically uses a maximum spanning tree (MST) for decoding, which decomposes the dependency tree into small factors, such as edge dependencies, and scores the entire tree by summing the scores of all included factors. The parser determines the relationship between words by finding the optimal tree structure in the graph [15]. Different models are available for conducting graph-based dependency parsing. These include statistical models that use various approaches, such as auto subtrees [16], first-order [17], and higher orders [18,19], as well as neural models that utilize techniques like CNN [20], LSTMs [21,22], and GNN [5]. Graph-based models offer a

^{*} Corresponding author.

E-mail addresses: irkham@ugm.ac.id (I. Huda), mediw@ugm.ac.id (M.E. Wibowo), pulungan@ugm.ac.id (R. Pulungan).

<https://doi.org/10.1016/j.neucom.2025.130482>

Received 14 June 2024; Received in revised form 14 March 2025; Accepted 7 May 2025

Available online 22 May 2025

0925-2312/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

more structured and flexible learning approach than transition-based models, resulting in higher accuracy. However, they tend to require longer training times due to computations associated with parse trees and their potential combinations, making them time-consuming and computationally expensive [23].

Li et al. [5] proposed a graph-based SDP model using graph neural networks that was more accurate than others. This model performs well but relies on a static, error-prone biaffine parser. Errors might accumulate if construction concerns are not addressed before graph representation learning. Following their prior work [5], Li et al. [24] suggested an upgraded GNN-based model. This updated model's dynamic graph learning framework combines graph creation to create an initial graph with graph representation learning (GNN). This framework enables the model to learn the graph's structure and representations simultaneously, improving accuracy and lowering errors. Graph structure learning (GSL) uses two MLPs and a biaffine attention network during graph creation. Both studies use graph convolutional networks (GCN) and graph attention networks (GAT) for graph representation learning.

GNN is the most prominent approach in recent research related to graph learning and has proven effective in various tasks, such as node prediction, link prediction, and graph classification. GNN takes the original features and adjacency matrix as input and outputs node embeddings as a graph representation [5]. GNN has various variants in its implementation, such as GCN [25], GAT [26], GATv2 [27], GraphSAGE [28], and graph isomorphism networks (GIN) [29]. Brody et al. [27] reported that the GATv2 model has the best accuracy for the GNN model with attention, while for the non-attention GNN model, GIN has the best accuracy. While GNN can learn powerful representations of graphs, their effectiveness is heavily influenced by the quality of the graph structure used. Recent research has highlighted that unreliable graph structures can significantly hinder GNN's representation capabilities. Therefore, exploring suitable graph structures for GNNs to achieve robust and reliable graph representations is crucial. To address this challenge, GSL was developed to extract useful graph structures from data for learning better graph representations with GNN [30].

In Li et al. [24], GSL generates an adjacency matrix based only on feature similarity by measuring the distance between two nodes using a metric learning function, implemented with two multilayer perceptrons and a biaffine attention network. In real-world applications, as explored by Zou et al. [30], graph generations obey some underlying principles, including not only feature similarity (homophily) but also sparsity and degree distribution. Zou et al. [30] implemented the DGSLN method that fulfills these three principles. This method fulfills the principle of homophily through adaptive structural learning with an attention mechanism and addresses sparsity with differentiable graph sparsification. Without sparsity, we may end up with an overly dense graph with a high number of edges, leading to increased computational complexity, noisy data, and overfitting [31]. Furthermore, the method also addresses degree distribution by employing gated graph integration to balance the contributions of the initial and learned graphs effectively, as well as implementing regularization techniques for a hybrid loss function. Ignoring the principle of degree distribution may result in unrealistic graph structures and the loss of important nodes [32]. The DGSLN method has been proven to improve GNN performance by providing a quality initial graph structure, resulting in more accurate predictions and classifications. Considering this, DGSLN has the potential to be applied to other tasks that necessitate good initial graphs, such as SDP.

In this study, we propose HybridGL, a GNN-based SDP model based on Li et al.'s dynamic learning graph architecture [24], which has demonstrated promising results as the state-of-the-art approach. In contrast to Li et al. [24], whose GSL stages are based primarily on feature similarity (homophily principle), we adapt the DGSLN technique, which meets the three graph-generation principles. This adaptation is required to improve GNN performance and prevent graph generation difficulties caused by broken principles. We use the hybrid loss with

graph regularization of Zou et al. [30] to see whether it improves the learned graph's performance. Our study also shows that DGSLN is effective in link prediction tasks like SDP, unlike Zou et al. [30], which concentrated on node and graph categorization. We also set a threshold for the predicted adjacency matrix at the end of the GSL stage to make it appropriate for graph representation learning in the dynamic learning graph architecture. To boost model performance during graph representation learning, we employ GNN variants GIN and GATv2, which outperform Li et al.'s GCN and GAT [24].

2. Related work

2.1. Semantic dependency parsing

Semantic dependency parsing (SDP) identifies semantic correlations between words in phrases, revealing fundamental meaning relationships. SDP illustrates relationships with semantic graphs [15]. Initial dependency parsing uses projective or non-projective trees; mostly, dependency trees are syntactic-oriented [33]. Many works on dependency parsing over graphs focus on semantic interactions between words [34].

Recently, significant developments have been made in the SDP models. These models can be categorized into two main types: transition-based and graph-based. Initial work for feature-rich transition-based dependency parsing was suggested by Nivre [35], where the transition actions include shift, arc-left, and arc-right operations. Later, another transition configuration proposed by Gómez-Rodríguez and Nivre [36] was based on multiplanarity, which can handle various complex cases, such as non-projective dependencies. Yuan et al. [37] introduced a bidirectional approach to transition-based SDP, where the parser learns from both left-to-right and right-to-left directions, which differs from the traditional uni-directional method. Fernández-González and Gómez-Rodríguez developed a pointer network-based transition-based parser that selects transitions between ROOT, Attach-p, and Shift [7]. Fernández-González and Gómez-Rodríguez [38] also developed a transition-based parser with a bottom-up hierarchical pointer network orientation, offering three alternatives: left-to-right, right-to-left, and outside-in.

Graph-based parsers are more complicated but more accurate than transition-based ones. Transition-based parsing is faster and simpler but more error-prone. Graph-based approaches utilize the MST algorithm for decoding purposes, and the initial work for this approach is the first-order MST proposed by McDonald et al. [17] based on the discriminative max-margin training. Later, other works have also explored higher-order MST parsers, such as second-order MST [39], third-order MST [18], and fourth-order MST [19]. Furthermore, Ji et al. [40] developed another graph-based parser utilizing GNNs to capture high-order information in the graphs effectively. Li et al. [5] developed a graph-based parser that utilizes GNN to encode high-order information to increase SDP accuracy, and a biaffine parser [41] was used to form the initial matrix. Li et al. [24] developed a graph-based parser that uses GNN but replaces the initial matrix with GSL, improving prior research. They employed a dynamic graph learning framework instead of a static biaffine parser to integrate graph building and representation to reduce error accumulation.

2.2. Graph structure learning

A popular topic in graph learning is graph structure learning (GSL), which optimizes initial graphs for representation learning. Three basic GSL methods are metric-based, neural-based, and direct. Metric-based techniques calculate node features or embedding similarity using kernel functions. This similarity is used as edge weights based on the network homophily assumption that edges connect similar nodes. The initial work on metric-based approaches is in adaptive graph convolutional neural networks (AGCN) by Li et al. [42], where they introduced a structure learning model based on distance metric learning. AGCN

computes the generalized Mahalanobis distances between each pair of node features. Zhao et al. [43] introduced a method for calculating edge weights on heterogeneous graphs using cosine similarity. Their approach combines feature similarity graphs and metapath-induced semantic graphs to obtain more comprehensive results. In their study, Yuan et al. [44] proposed a method to improve the expressivity of graph representations by combining multiple kernels. Specifically, they utilized cosine kernels on node embeddings and incorporated a diffusion kernel to capture the graph structure's local and global characteristics.

In contrast to metric-based methods, neural-based approaches utilize complex deep neural networks to model edge weights based on node features and representations. The initial work of this approach, pioneered by Pilco and Rivera [45], utilizes a basic one-layer neural network to understand graph structures using local and global node embeddings iteratively. Similarly, graph learning-convolutional networks (GLCN), proposed by Jiang et al. [46], employ a single-layer neural network to encode the pairwise relationships between two node embeddings. Li et al. [24] introduced a neural-based GSL module for SDP prediction in their dynamic graph learning framework. The GSL module uses two MLPs to represent the start and finish nodes and a biaffine attention network to score possible edges. Zou et al. [30] developed a neural-based GSL module for their differentiable structure learning approach. They learned graph node pairwise dependencies using an attention technique and the LeakyReLU activation function.

Furthermore, direct methods differ from the previously mentioned approaches as they consider the target graph's adjacency matrix as free learning parameters. These methods do not depend on node representations to determine edge connections, offering greater flexibility but facing challenges in optimizing the matrix parameters [47]. Most direct approaches optimize the adjacency matrix using graph regularization techniques, such as Jin et al. [48], who developed ProGNN that optimizes a given graph by employing sparsity, low rank, and smoothness. Duan et al. [49] also used a direct approach to form the adjacency matrix with causality-based measurements for transfer entropy among its variables and then thresholded the transfer entropy values to obtain edges in the graph.

In later developments, several studies have developed GSL with a hybrid approach by combining several types of GSL approaches to leverage the advantages of each type. Duan et al. [50], as part of their MTPool framework, developed a metric-based GSL with a similarity matrix for distance measurement of dependency relationships, but then combined it with a neural-based approach using a learnable model and activation function to make the model more adaptive and robust. An et al. [51] also used a hybrid approach in their two graph learners. In the first graph learner, if the topology information is unknown, a full graph parameterization method is used, along with two metric-based learners, namely attention and multilayer perceptron (MLP). In the second graph learner, if the topology information is known, a graph learner based on GNN is used. Zou et al. [30] introduced DGSLN, a hybrid approach that uses metric-based ideas to compare node features but fully integrates them into neural network architecture with attention, graph sparsification, and gated integration. They then optimized the algorithm using graph regularization, like feature smoothness and property constraints, and hybrid loss functions.

2.3. Graph neural networks

Graph neural networks (GNN) are a type of neural network designed to process and analyze graph-structured data. The initial inspiration for GNNs stems from the longstanding use of neural networks for graph analysis. Recursive neural networks were first utilized on directed acyclic graphs by Sperduti and Starita [52] and Frasconi et al. [53]. Afterward, recurrent neural networks and feedforward neural networks were introduced into the literature by Scarselli et al. [54] and Micheli [55] to tackle cycles. Recent progress in deep neural networks, particularly convolutional neural networks (CNNs) [56], has led to the

rediscovery of GNNs. CNNs can extract multi-scale localized spatial features and compose them to construct highly expressive representations of input data. This inspired the original GNN models proposed by Scarselli et al. [54] and Merkwirth and Lengauer [57], which were based on the message-passing concept. Furthermore, Kipf and Welling [25] generalized and united prior approaches into an approach known as graph convolutional networks.

Two primary approaches are used in GNNs to process and learn from graph-structured data: spectral and spatial [58]. The spectral approach is based on graph signal processing and utilizes the spectral theory of graphs. It applies a Fourier transform to project the graph signals (node features) to the spectral domain and performs convolution operations in it. Graph convolutional network (GCN) [25], one of the widely used spectral-based GNNs, simplifies the convolution operation on graph-structured data using a localized approximation of spectral filters. The adaptive GCN introduced by Li et al. [42] was proposed to learn the underlying relations between nodes in a graph. It learns a 'residual' Laplacian graph and adds it to the original Laplacian matrix, which has proven effective in several graph-structured datasets.

The spatial approach, however, concentrates on capturing the underlying neighborhood structure of each node in the graph. It models local interactions between nodes by directly applying convolution operations based on the graph's topology. The diffusion convolutional neural network (DCNN) by Atwood and Towsley [59] is a spatial-based GNN that uses transition matrices to define the neighborhood structure and performs convolutions based on these matrices. The graph attention network (GAT) proposed by Veličković et al. [26] is a spatial-based GNN that utilizes attention mechanisms to determine the significance of each neighboring node during aggregation. To overcome the limitations of earlier versions, Brody et al. [27] introduced GATv2 with an enhanced and dynamic attention mechanism, which increases robustness to noise and improves performance for representation learning tasks on graphs. Graph isomorphism network (GIN) introduced by Xu et al. [29] is another spatial-based GNN designed to model injective multiset functions for neighbor aggregation. It is as powerful as the Weisfeiler-Lehman graph isomorphism test to capture complex graph structures.

3. Preliminaries

3.1. Differentiable structural learning neural network (DGSLN)

DGSLN is a graph structure learning (GSL) built upon the work of Zou et al. [30]. It incorporates an attention mechanism to effectively learn a flexible graph structure from the properties of nodes in each layer, which enables the network to learn robust representations of graphs. It consists of multiple layers and three primary procedures: adaptive structural learning, differentiable graph sparsification, and gated graph integration.

3.1.1. Adaptive structural learning

Adaptive structural learning is a process that involves dynamically capturing pairwise node relationships through a self-attention mechanism. This process is grounded in the principle of homophily, which suggests that similar nodes are more likely to be connected in a graph. The adaptive structural learning approach constructs a basic graph structure by measuring the similarity among nodes. The attention mechanism, proven to be an efficient learnable similarity metric, captures the relational importance within the feature space and explores the underlying graph structure. Let V be the set of n nodes, each with d features. Given the sets of node features $\mathbf{X}^{(\ell)} \in \mathbb{R}^{n \times d}$ at the ℓ -th layer as input, the attention mechanism is applied to learn the pairwise dependency relationship of the graph nodes. In particular, $\alpha_{i,j}^{(\ell)}$, the dependency between nodes v_i and v_j , is computed by:

$$\alpha_{i,j}^{(\ell)} = \text{LeakyReLU} \left(\theta^T (\mathbf{W}_i^{(\ell)} \oplus \mathbf{W}_j^{(\ell)}) \right), \quad (1)$$

where $\tilde{x}_i^{(\ell)}$ and $\tilde{x}_j^{(\ell)}$ are the feature vectors of nodes v_i and v_j , respectively. These features need to be transformed using a shared linear transformation, parameterized by the weight matrix $\mathbf{W} \in \mathbb{R}^{d \times d}$. Operator $\oplus$ concatenates the two transformed feature vectors, and $\Theta \in \mathbb{R}^{2d \times 1}$ is the learnable weight or shared attentional vector. The activation function used is LeakyReLU, which introduces non-linearity to the model to capture relationships and dependencies between nodes effectively. LeakyReLU mitigates the dying ReLU issue (outputting zero for any negative input) by allowing a small gradient when the unit is inactive (i.e., for negative input values), ensuring that all neurons have the opportunity to update and learn during training [60,61]. The use of LeakyReLU contributes to the robustness of the learned graph representations by ensuring the model can learn meaningful information even from less prominent or negative feature interactions.

The LeakyReLU function is applied in an attention mechanism that computes the importance of edges between nodes in a graph (dependency). The softmax function is then used to normalize all $\alpha_{i,j}^{(\ell)}$ to ensure they form a valid probability distribution, namely:

$$S_{i,j}^{(\ell)} = \text{softmax}(\alpha_{i,j}^{(\ell)}) = \frac{\exp(\alpha_{i,j}^{(\ell)})}{\sum_{k=1}^n \exp(\alpha_{i,k}^{(\ell)})}. \quad (2)$$

3.1.2. Differentiable graph sparsification

Differentiable graph sparsification is designed to convert densely connected graphs into sparse graphs efficiently. Since the output of the softmax function in the previous step always generates non-zero values, the learned graph becomes densely connected. A sparse attention masking operation is applied to convert the dense graph into a sparse one. This operation involves selecting a threshold, i.e., the k th largest element in each row of $\alpha_{i,j}^{(\ell)}$, gathering them to form vector $\tilde{\tau} = [\tau_1, \tau_2, \dots, \tau_n]^T$, and filtering out connections that fall below the threshold. The masking function $\tilde{\alpha}_{i,j}^{(\ell)} = M(\alpha_{i,j}^{(\ell)}, k)$ is defined by:

$$\tilde{\alpha}_{i,j}^{(\ell)} = M(\alpha_{i,j}^{(\ell)}, k) = \begin{cases} \alpha_{i,j}^{(\ell)}, & \alpha_{i,j}^{(\ell)} \geq \tau_i, \\ -\infty, & \text{otherwise,} \end{cases} \quad (3)$$

where k is a hyperparameter, and τ_i is the k th largest value of row i . The output graph structure $\tilde{S}_{i,j}^{(\ell)}$ can then be computed by:

$$\tilde{S}_{i,j}^{(\ell)} = \text{softmax}(M(\alpha_{i,j}^{(\ell)}, k)). \quad (4)$$

3.1.3. Gated graph integration

Gated graph integration incorporates the learned graph structure into the raw graph structure. This integration involves a gated fusion mechanism, which calculates weight factors for each graph and merges them using a weighted sum. This integration step is essential in creating an optimized graph that adjusts to the current layer of the network. The formal definition of gated fusion is given by:

$$\mathbf{A}^{(\ell)} = \tilde{\mathbf{W}}_s \odot \tilde{\mathbf{S}}^{(\ell)} + \tilde{\mathbf{W}}_a \odot \mathbf{A}^{(\ell-1)}, \quad (5)$$

where $\odot$ denotes the element-wise (Hadamard) product, $\tilde{\mathbf{W}}_s$ and $\tilde{\mathbf{W}}_a \in \mathbb{R}^n$ are learnable parameters that determine the influence degree of $\tilde{\mathbf{S}}^{(\ell)}$ and $\mathbf{A}^{(\ell-1)}$, respectively, on the new graph topology, $\tilde{\mathbf{S}}^{(\ell)}$ is the learned graph structure, $\mathbf{A}^{(\ell-1)}$ is the graph structure of the previous layer, and $\mathbf{A}^{(\ell)}$ is the fused optimal graph of the current layer.

3.2. Graph isomorphism network (GIN)

GIN was developed to create a GNN variant with discriminative or representational power in distinguishing graph structures equivalent to that of the Weisfeiler-Lehman (WL) test [29]. When function $\text{GNNLayer}(\cdot, \cdot)$ is implemented in GIN, the representation $\tilde{r}_i^{(\ell)}$ of node v_i at the ℓ -th layer is given by:

$$\tilde{r}_i^{(\ell)} = \text{MLP}^{(\ell)} \left((1 + \epsilon^{(\ell)}) \tilde{r}_i^{(\ell-1)} + \sum_{j \in \mathcal{N}(i)} \tilde{r}_j^{(\ell-1)} \right), \quad (6)$$

where $\epsilon^{(\ell)}$ is a learnable parameter that allows mixing the features of the node itself with its neighbors' features, $\mathcal{N}(i)$ is the set of node v_i 's neighbors, and $\tilde{r}_i^{(\ell-1)}$ is the contextual representation of node v_i and $\tilde{r}_i^{(0)} = \tilde{x}_i$. $\text{MLP}^{(\ell)}$ represents a multilayer perceptron applied at the ℓ -th layer. It is a key component that allows for the powerful representational capacity of the GIN and is used to model injective multiset functions for neighbor aggregation.

3.3. Graph attention network version 2 (GATv2)

The main difference between GATv2 and the original GAT lies in the attention mechanism. GAT computes a static kind of attention, where the ranking of attention scores is unconditioned on the query node. On the other hand, GATv2 introduces a dynamic attention mechanism, which is more expressive and potentially enables the detection of more intricate patterns within the data. The key change in GATv2 is a simple modification in the order of internal operations in the attention function of GAT. When function $\text{GNNLayer}(\cdot, \cdot)$ is implemented in GATv2, the representation $\tilde{r}_i^{(\ell)}$ of node v_i at the ℓ -th layer is given by:

$$\tilde{r}_i^{(\ell)} = \alpha_{i,i}^{(\ell)} \mathbf{W}^{(\ell)} \tilde{r}_i^{(\ell-1)} + \sum_{j \in \mathcal{N}(i)} \alpha_{i,j}^{(\ell)} \mathbf{W}^{(\ell)} \tilde{r}_j^{(\ell-1)}, \quad (7)$$

where $\alpha_{i,j}^{(\ell)}$, the attention coefficient determining the importance of node v_j 's features to node v_i at this specific layer ℓ , is given by:

$$\alpha_{i,j}^{(\ell)} = \frac{\exp \left(\tilde{a}^{(\ell)\top} \text{LeakyReLU} \left(\mathbf{W}^{(\ell)} \tilde{r}_i^{(\ell-1)} + \mathbf{W}^{(\ell)} \tilde{r}_j^{(\ell-1)} \right) \right)}{\sum_{k \in \mathcal{N}(i) \cup \{i\}} \exp \left(\tilde{a}^{(\ell)\top} \text{LeakyReLU} \left(\mathbf{W}^{(\ell)} \tilde{r}_i^{(\ell-1)} + \mathbf{W}^{(\ell)} \tilde{r}_k^{(\ell-1)} \right) \right)}, \quad (8)$$

where $\mathbf{W}^{(\ell)}$ and $\tilde{a}^{(\ell)}$ are the learned weight matrix and attention vector, respectively, and $\tilde{r}_i^{(\ell-1)}$ is the contextual representation of node v_i and $\tilde{r}_i^{(0)} = \tilde{x}_i$. Index k is used to sum over all neighbors of v_i (including v_i itself if self-attention is included) to normalize the attention coefficients to sum up to 1.

3.4. Dependency relationship learning

The dependency relationship learning module utilizes the biaffine attention-based parser [41], which consists of two components: edge prediction and label prediction. The graph node representation $\tilde{r}_i$ and the contextualized representation $\tilde{x}_i$ from the previous stage are inputs to the biaffine attention mechanism. To represent each node v_i , $\tilde{r}_i$ and $\tilde{x}_i$ are concatenated. Then, an MLP is used to split the final node representation $\tilde{z}_i$ into a start node representation and an end node representation for both components, namely:

$$\tilde{z}_i = \tilde{r}_i \oplus \tilde{x}_i, \quad (9)$$

$$h_i^{\text{edge-start}} = \text{MLP}^{\text{edge-start}}(\tilde{z}_i), \quad (10)$$

$$h_i^{\text{label-start}} = \text{MLP}^{\text{label-start}}(\tilde{z}_i), \quad (11)$$

$$h_i^{\text{edge-end}} = \text{MLP}^{\text{edge-end}}(\tilde{z}_i), \quad (12)$$

$$h_i^{\text{label-end}} = \text{MLP}^{\text{label-end}}(\tilde{z}_i), \quad (13)$$

where $h_i^{\text{edge-start}}$, $h_i^{\text{edge-end}}$, $h_i^{\text{label-start}}$, and $h_i^{\text{label-end}}$ are the representations of v_i in the respective nodes.

To predict the edges and labels, two biaffine classifiers are utilized:

$$s_{i,j}^{\text{edge}} = \text{Biaff}^{\text{edge}}(h_i^{\text{edge-end}}, h_j^{\text{edge-start}}), \quad (14)$$

$$s_{i,j}^{\text{label}} = \text{Biaff}^{\text{label}}(h_i^{\text{label-end}}, h_j^{\text{label-start}}), \quad (15)$$

where $s_{i,j}^{\text{edge}}$ and $s_{i,j}^{\text{label}}$ are the scores of the edge and label between the nodes v_i and v_j , respectively.

To prevent the generation of invalid trees in the syntactic dependency parsing, we employ the minimum-spanning-tree-based algorithm

of Dozat and Manning [22]. Finally, the outputs of the edge and label predictions are formulated by:

$$\tilde{y}_{i,j}^{edge} = s_{i,j}^{edge} > 0, \quad (16)$$

$$\tilde{y}_{i,j}^{label} = \arg \max s_{i,j}^{label}, \quad (17)$$

where $\tilde{y}_{i,j}^{edge}$ and $\tilde{y}_{i,j}^{label}$ are the outputs of the edge and label prediction components, respectively.

3.5. Model learning and hybrid loss function

The training process involves backpropagating errors to the parser after summing the losses of the three modules: edge, label, and adjacency matrix. The loss functions of the three prediction modules are defined based on the general cross-entropy function:

$$CE(p, q) = - \sum_x p(x) \log(q(x)), \quad (18)$$

namely:

$$\mathcal{L}_{edge}(\theta_1) = CE(\tilde{y}_{i,j}^{edge}, y_{i,j}^{edge}), \quad (19)$$

$$\mathcal{L}_{label}(\theta_2) = CE(\tilde{y}_{i,j}^{label}, y_{i,j}^{label}), \quad (20)$$

$$\mathcal{L}_{adj}(\theta_3) = CE(\tilde{A}_{i,j}, A_{i,j}), \quad (21)$$

where θ_1 , θ_2 , and θ_3 are their respective learned parameters. Afterward, we backpropagate the error to update the parser.

For the adjacency matrix (GSL) module, Zou et al. [30] asserted that a hybrid loss function can be employed to optimize the learned graph's quality. Therefore, in addition to using cross-entropy as the loss function related to graph structure prediction, a regularization technique is also incorporated. These techniques are employed to steer the learning process of the graph structure, ensuring that the resulting graph meets specific attributes, such as sparsity and connectivity. In this case, the hybrid loss function of the adjacency matrix is formally defined by:

$$\mathcal{L}_{adj+reg} = \mathcal{L}_{adj}(\theta_3) + \sum_{\ell} \mathcal{L}_{reg}^{(\ell)}, \quad (22)$$

where $\mathcal{L}_{reg}^{(\ell)}$ is the regularization function used to optimize the graph structure at layer ℓ . The regularization function consists of two terms to control feature smoothness and property constraint of the learned graph $\tilde{\mathbf{S}}^{(\ell)}$ (cf. Eq. (27)).

In order to optimize the overall summed loss function, we employ the adaptive moment estimation (Adam) method. Adam is a straightforward and computationally efficient approach designed to optimize stochastic objective functions using gradient-based methods, which is easy to implement and has low memory requirements [62]. Adam is preferred for its resilience in managing many types of objective functions and its capacity to effectively handle extensive datasets or parameters. The summed loss function of the three modules $\mathcal{L}$ can then be expressed as:

$$\mathcal{L} = \alpha \mathcal{L}_{adj+reg} + \beta \mathcal{L}_{edge} + (1 - \alpha - \beta) \mathcal{L}_{label}, \quad (23)$$

where α and β are two tunable interpolation constants and $0 < \alpha + \beta < 1$.

3.5.1. Feature smoothness

Feature smoothness is a metric reflecting the similarity of node representations. The first regularization term aims to control feature smoothness. Feature smoothness constraints can be imposed in the objective function to ensure the learned graph structure $\tilde{\mathbf{S}}^{(\ell)}$ adapts to the node features $\tilde{\mathbf{x}}$. The first regularization is defined by:

$$\mathcal{L}_f^{(\ell)} = \sum_{\forall v_i, v_j \in V} \phi(\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}_j), \quad (24)$$

where $\mathcal{L}_f^{(\ell)}$ is the feature smoothness loss function to be minimized, ϕ is a metric function that measures the difference among nodes. Typical measures, for instance square error, can be adopted, namely:

$$\sum_{\forall v_i, v_j \in V} \phi(\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}_j) = \sum_{\forall v_i, v_j \in V} \|\tilde{\mathbf{x}}_i - \tilde{\mathbf{x}}_j\|_F^2 \tilde{\mathbf{S}}_{i,j}^{(\ell)} = \text{tr}(\mathbf{X}^T \mathbf{L} \mathbf{X}), \quad (25)$$

where $\|\cdot\|_F$ is the Frobenius norm, $\text{tr}(\cdot)$ is the trace of a matrix, $\mathbf{L} = \mathbf{D} - \tilde{\mathbf{S}}^{(\ell)}$ is the graph Laplacian matrix, and $\mathbf{D}$ is the degree matrix.

3.5.2. Property constraint

The learned graph is constrained to satisfy more graph structure properties (such as sparsity and connectivity). The second regularization term aims to explore more prior information, as defined by:

$$\mathcal{L}_p^{(\ell)} = -\lambda_1 \tilde{\mathbf{I}}^T \log(\tilde{\mathbf{S}}^{(\ell)} \tilde{\mathbf{I}}) + \frac{\lambda_2}{2} \|\tilde{\mathbf{S}}^{(\ell)}\|_F^2, \quad (26)$$

where $\tilde{\mathbf{I}} = [1, \dots, 1]^T$ and $\tilde{\mathbf{S}}^{(\ell)} \tilde{\mathbf{I}}$ is the node degree vector. The first term applies the logarithmic barrier on $\tilde{\mathbf{S}}^{(\ell)} \tilde{\mathbf{I}}$, which forces the degree to be positive but does not prevent the edges from being 0.

To summarize, the total regularization loss can be defined by:

$$\mathcal{L}_{reg}^{(\ell)} = \lambda_0 \mathcal{L}_f^{(\ell)} + \mathcal{L}_p^{(\ell)}, \quad (27)$$

where the pre-defined hyperparameters λ_0 , λ_1 , and λ_2 determine the impact of the regularization terms on the overall task.

4. Proposed method

This study attempts to improve SDP models for predicting the semantic dependency graphs (SDGs) of English sentences. We use the best SDP framework, dynamic graph learning [24], to do this. Then, we generate the adjacency matrix using hybrid GSL (HGSL), an adaptation of DGSLN [30], to get the initial structure for graph representation learning (GNN) and adapt its hybrid loss functions for graph optimization. We employ GIN and GATv2, the finest GNN variations, for graph representation learning. Fig. 1 depicts the architecture of our model, the HybridGL.

Given a sentence s consisting of n words $\{w_1, w_2, \dots, w_n\}$, there are four stages to predicting the shape of the SDG, i.e.:

1. Contextualized representation learning: The BiLSTM model is used to contextualize sentence words.
2. Hybrid graph structure learning: A neural network-based HGSL module learns the adjacency matrix of potential SDGs.
3. Robust graph representation learning: Two robust GNN variants learn expressive node representation from words' contextual representation and learned adjacency matrix.
4. Dependency relationship learning: The biaffine attention-based model uses expressive node representations and contextualized representations to examine the words' dependency relationship, expressed through edges and labels.

4.1. Contextualized representation learning

In order to obtain a contextualized representation, it is necessary to concatenate the word embedding and feature embedding before feeding them into a bidirectional LSTM, namely by:

$$\tilde{\mathbf{c}}_i = \tilde{\mathbf{e}}_i^{word} \oplus \tilde{\mathbf{e}}_i^{feat}, \quad (28)$$

$$\tilde{\mathbf{x}}_i = \text{BiLSTM}(\tilde{\mathbf{c}}_i), \quad (29)$$

where $\tilde{\mathbf{c}}_i$ represents the concatenation ($\oplus$) of the *word* and *feature* embeddings for the word w_i , while $\tilde{\mathbf{x}}_i$ is the contextualized representation of word w_i .

The word embeddings from GloVe [63], which is of 100 dimensions, is used for the English dataset. Four types of feature embeddings are used: (1) A randomly generated part-of-speech (POS) tag embedding,

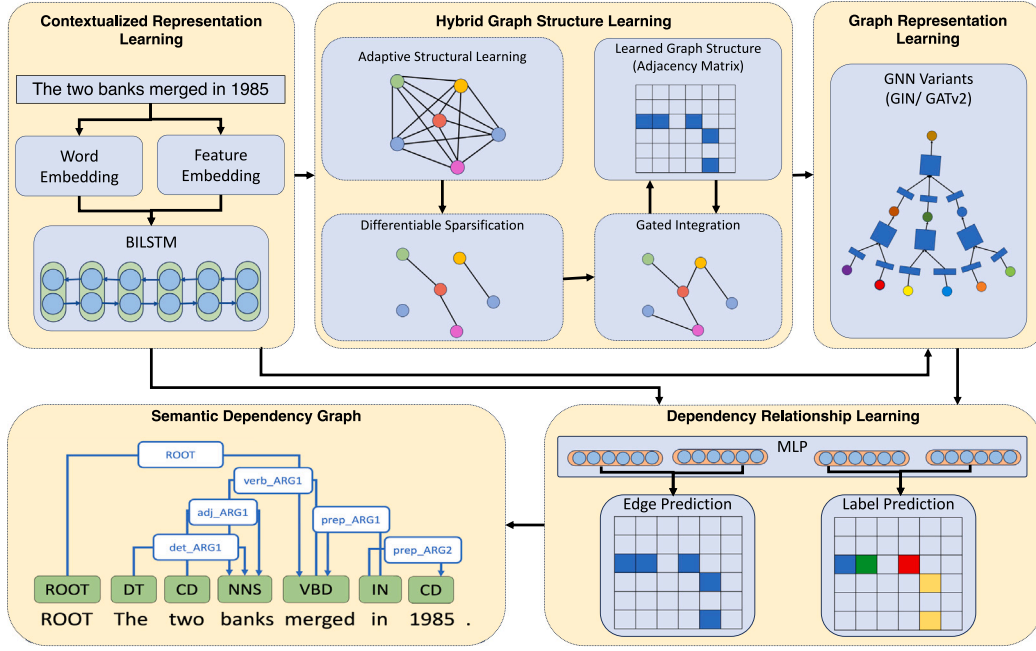


Fig. 1. Architecture of the proposed HybridGL.

defined as $\mathbf{E}^{pos} \in \mathbb{R}^{n \times d}$, where n is the number of POS tags and d is the embedding dimension. (2) A character embedding that captures details within each token via a CharLSTM that convolves over three-character sequences at each step. (3) A randomly generated lemma embedding $\mathbf{E}^{lemma} \in \mathbb{R}^{m \times d}$, where m is the number of lemmas. (4) The bidirectional encoder representation from a Transformer (BERT) embedding, extracted from a pre-trained BERT model. The output is a representation vector $\tilde{x} \in \mathbf{X}$ with a total dimension d , which affects the number of hidden states and the number of layers used as parameters in the BiLSTM.

4.2. HGSL to generate the adjacency matrix

The graph structure learning (GSL) approach is employed to construct the adjacency matrix ($\tilde{\mathbf{A}}$) for graph representation learning. SDP nodes represent words, and a graph reflects sentence semantic dependencies in GSL. The GSL problem can be formulated as $G = (\mathbf{A}, \mathbf{X})$, where G is the graph, $\mathbf{A}$ is the initial graph structure of the adjacency matrix, and $\mathbf{X}$ is the contextualized representation vector of words and node features. GSL simultaneously learns the best graph topology to boost GNN representation. HGSL, an adaptation of DGSLN [30], is used in this study, whose structure is depicted in Fig. 2.

HGSL consists of multiple layers (this study uses one to three layers) and four primary processes:

1. Adaptive structural learning: The ℓ -th layer receives node attributes $\mathbf{X}^{(\ell)}$ and employs self-attention to dynamically determine pairwise node relationships. The output is the learned densely connected graph $\mathbf{S}^{(\ell)}$.
2. Differentiable graph sparsification: $\mathbf{S}^{(\ell)}$ is provided as input at the ℓ -th layer and transforms it into a sparse graph $\tilde{\mathbf{S}}^{(\ell)}$ using an explicit masking function.
3. Gated graph integration: It takes a sparse graph $\tilde{\mathbf{S}}^{(\ell)}$ and an initial graph $\mathbf{A}^{(\ell-1)}$ from the preceding layer as input. Then, it adaptively combines the two input graphs using a gating function to obtain revised graph structure $\mathbf{A}^{(\ell)}$.
4. Threshold limit: In the last stage of HGSL, we use the optimal graph $\mathbf{A}$ by setting a threshold limit t to obtain the estimated adjacency matrix $\tilde{\mathbf{A}}$.

To get the estimated adjacency matrix $\tilde{\mathbf{A}}$, we use the optimal graph $\mathbf{A}$ by specifying a threshold limit t . A directed edge from w_i to w_j exists when $A_{i,j}$ exceeds the threshold t , namely:

$$\tilde{A}_{i,j} = \begin{cases} 1, & A_{i,j} > t, \\ 0, & A_{i,j} \leq t. \end{cases} \quad (30)$$

The output is an adjacency matrix $\tilde{\mathbf{A}}$ of dimensions n , representing a graph with a total of n nodes. The number of nodes corresponds to the number of words in the sentence.

4.3. GIN and GATv2 for graph representation learning

GNN variants are used to learn expressive graph node representation ($\mathbf{R}$). GNN variants learn from contextual representations of words and features ($\mathbf{X}$) and the adjacency matrix ($\tilde{\mathbf{A}}$) from HGSL to understand their graph structure. We utilize two of the best GNN variants, i.e., graph isomorphism network (GIN) and graph attention network version 2 (GATv2), which have been shown to be effective by Brody et al. [27].

The GNN variation consists of h layers, with $\mathbf{R}^{(0)} = \mathbf{X}$, and $\mathbf{R}^{(\ell)}$ at the ℓ -th layer, as a result, is formulated as:

$$\mathbf{R}^{(\ell)} = \text{GNNLayer}^{(\ell-1)}(\mathbf{R}^{(\ell-1)}, \tilde{\mathbf{A}}). \quad (31)$$

The learned adjacency matrix $\tilde{\mathbf{A}}$ consists of elements $\tilde{A}_{i,j}$ representing the presence (weight) of an edge from node v_i to node v_j . We use GNN with h layers to capture information by aggregating h -order neighborhoods, allowing the model to capture and integrate local graph structure into the learning process. The adjacency matrix determines which nodes are neighbors, which is essential because the attention mechanism in GATv2 operates over these neighbors to compute the weighted sum of their features. Also, it is crucial for the aggregation step in GIN's update rule.

The output $\mathbf{R}^{(h)}$ has dimension $n \times d$, where n is the number of nodes (representing the number of words in the sentence), while d is the number of feature representations of each node. The value of d affects the number of hidden states used as a parameter in the GNN variants. The result, i.e., graph node representation $\mathbf{R}^{(h)}$, will be fed to the dependency relationship learning to predict the edges and edge labels using an MLP and the biaffine classifier.

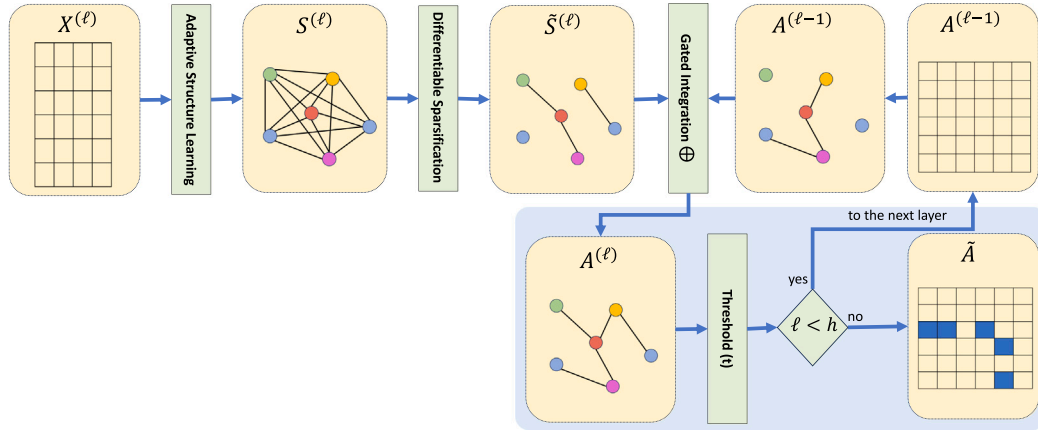


Fig. 2. The structure of HGSL is derived from DGSLN, consisting of three primary processes: adaptive structure learning, differentiable graph sparsification, and gated graph integration with the addition of a threshold for the adjacency matrix.

5. Experiments

We conduct experiments to evaluate the performance of our proposed hybrid model of semantic dependency parsing HybridGL in producing a labeled graph representing the semantic connections between words within a sentence.

5.1. Datasets

Semantic dependency parsing experiments are conducted to evaluate the proposed model, utilizing the SemEval 2015 Task 18 dataset [34]. This dataset encompasses three languages: English, Chinese, and Czech. For English, the dataset includes three distinct formalisms: DELPH-IN MRS (DM) [64], Predicate-Argument Structure (PAS) [65], and Prague Semantic Dependencies (PSD) [66]; for Czech, only the PSD formalism is available; and for Chinese, only the PAS formalism.

The dataset used in this research is only English dataset, which consists of 33,964 training sentences from Sections 00-19 of the Wall Street Journal (WSJ) corpus, 1,692 development sentences from Section 20, 1,410 sentences from Section 21 as the in-domain (ID) test data, and 1,849 sentences sampled from the Brown corpus as the out-of-domain (OOD) test data.

5.2. Hyperparameters

The hyperparameter settings used in the proposed model for word embedding, feature embedding, LSTM, and GNN follow Li et al. [24]. For the number of GNN layers, we conduct experiments with 1 or 2 layers. HGSL hyperparameter settings, i.e., k , λ_0 , λ_1 , and λ_2 , also follow the optimum settings of DGSLN [30]. For the number of HGSL layers, we use the DGSLN setting, i.e., 3 layers, but also experiment with 1 and 2 layers. Furthermore, our experiments indicate that the optimum threshold t is 0.4–0.5.

The proposed model is optimized using Adam with the standard hyperparameter settings proposed by Li et al. [24] for decay rate, decay step length, warmup, parameters for controlling the exponential decay rates (β_1, β_2), and two tunable interpolation constants to optimize the loss function (α, β). We set the number of epochs to 200. An early-stop approach is implemented to prevent overfitting during training: if the validation loss does not improve within 30 epochs, the training is halted. The training also uses an LSTM learning rate of 0.001. The complete hyperparameter configuration details are listed in Table 1.

5.3. Experiment environment

The details of the hardware and software used in our experiments are listed in Table 2.

Table 1

Final hyperparameter configuration.

Layer	Hyperparameter	Value
Word embedding	English	100
	POS, Char, Lemma, BERT	100
	LSTM	3
	Hidden size	400
GNN	Dropout rate	0.33
	GCN/GIN/GATv2 layers	3
	GCN/GIN/GATv2 hidden size	600
	GCN/GIN/GATv2 dropout rate	0.33
HGSL	HGSL layers h	2–3
	k	4
	t	0.4–0.5
	λ_0	0.0005
Trainer	λ_1	0.1
	λ_2	0.01
	Optimizer	Adam
	LSTM learning rate	0.001
	Adam (β_1, β_2)	(0.9, 0.95)
	Decay rate	0.75
	Decay step length	5,000
	Warmup	0.2
	Loss (α, β)	(0.2, 0.2)

Table 2

Hardware and software used in the experiments.

Hardware/Software	Version
Hardware	
CPU	Intel(R) Core(TM) i5-13400F @ 2.50 GHz (16-core)
GPU	NVIDIA GeForce RTX 4070 SUPER
RAM	32 GB
Software	
Python	3.11.5
Pytorch	2.1.1
Anaconda	23.7.4
CUDA	12.3
IDE	Visual Studio Code v. 1.84.2

5.4. Evaluation metrics and baseline models

We employ the labeled F-measure (LF1) score, incorporating the ROOT edges, to assess the performance of our proposed model with parameters varying in the number of layers h , threshold t , hybrid loss, and feature embeddings on both the ID and OOD testing sets for each formalism, as well as to compute a macro-average across all three. The LF1 scores of the baseline methods and our model are averaged across five runs with varying random seeds, and the standard deviation of

Table 3
Summary of the baseline methods.

Method	Feature embedding	Adjacency matrix generation	Semantic parsing model
CompoSem	POS, Lemma, NE, BERT	–	Compositional neural semantic parser
FastSem	POS, Char, Lemma, NE, BERT	–	A* Apply Modify (AM) dependency parser
SemPointer	POS, Char, Lemma, BERT	–	Transition-based with pointer network
Flair	POS, Lemma, BERT, Flair	–	Biaffine attention parser
ACE-Fine-tune	Char, ELMo, BERT, Flair, XLM-R	–	Automated concatenation of embeddings
GNNSDP	POS, Char, Lemma, BERT	Biaffine parser	GNN (GCN, GAT)
DynGL	POS, Char, Lemma, BERT	MLP	GNN (GCN, GAT)

the model's LF1 scores is also computed. The baseline methods, whose details are listed in Table 3, are as follows:

1. CompoSem [6]: a compositional neural semantic parser that delivers good accuracies across various graph banks.
2. FastSem [67]: both an A* parser and a transition-based parser for AM dependency parsing, ensuring correctness and enhancing parsing speed.
3. SemPointer [7]: a model based on transition using the Pointer Network.
4. Flair [68]: employs not just BERT but also contextual string embeddings.
5. ACE-Fine-tune [69]: an enhanced model by incorporating an automated fusion of 11 different types of pre-trained embeddings into a biaffine attention-based parser.
6. GNNSDP [5]: a GNN-based parser that encodes higher-order information to improve SDP accuracy. It uses the biaffine parser to construct the initial matrix.
7. DynGL [24]: an improved version of GNNSDP that develops a GNN-based parser while replacing the formation of the initial matrix using MLP-based GSL. This research sets a new state-of-the-art for semantic dependency parsers.

6. Results and analysis

6.1. Impact of the number of layers and the adjacency matrix threshold

Almost all hyperparameters are set according to the settings that produced the best results in previous research and baseline models, as listed in Table 1. Two parameters, however, are peculiar to our proposed method: the number of HGSL layers h in the HybridGL model and the threshold t when generating the adjacency matrix. To assess the impact of these two parameters, we evaluate the LF1 scores of several variants of our proposed model. Our experiments indicate that the best configurations are obtained when h is set to 2 or 3 and t is set to 0.4 or 0.5. Table 4 shows the experiment results for only the relevant configurations out of all possible value permutations of the two parameters. The table lists the obtained average $\pm$ standard deviation LF1 scores for three variant models and the three (formalism) datasets (and the average of the three) by varying h and t with and without a hybrid loss function. A blue color entry indicates the better configuration for the specific variant on varying h and t , while the bold entry indicates the best variant (and configuration) for the corresponding dataset.

The table indicates that, generally, the proposed model performs better with 2 layers and a threshold of 0.5 than with 3 layers and a threshold of 0.4 for both ID and OOD data. The opposite occurs for the HybridGL model with the GIN variant without hybrid loss, in which case the proposed model with 3 layers and a threshold of 0.4 has better accuracy. Nevertheless, on average, most variants are better with 2 layers and a threshold of 0.5. Furthermore, the highest increase in the average accuracy is obtained by GATv2 without the hybrid loss variant, with a 0.081% increase for ID data, and GATv2 with the hybrid loss variant, with a 0.085% increase for OOD data.

6.2. Impact of the hybrid loss function

In DGSLN [30], incorporating a hybrid loss function impacts graph and node classification accuracy significantly. To determine whether a similar impact can be found in our study of semantic dependency parsing, we experiment with our proposed method, HybridGL, without hybrid loss (–HL) and with hybrid loss (+HL) function. The experiment results are also presented in Table 4, where an underlined entry indicates the better configuration (–HL or +HL) for the variant with the same h and t values.

The experiment results indicate that using the hybrid loss function generally does not improve accuracy but slightly decreases accuracy in both configurations, except for the GCN variant on the PSD dataset. This could happen because the employed hybrid loss functions, namely feature smoothness and property constraint, are more suitable for optimizing graph or node classification tasks, as done by Zou et al. [30], especially with graph homophily datasets. Link prediction tasks like SDP involve predicting relationships between words in a sentence that often connect dissimilar nodes (like a verb (e.g., walk) to a noun (e.g., David)), which is based on Platonov et al. [70], tends to be heterophilic. Hence, feature smoothness and property constraints are generally less suitable as optimization functions because they can cause over-smooth node embeddings, making it harder to distinguish between nodes and thus reducing link prediction accuracy. The best average accuracy is obtained by the GIN -HL variant for ID data and the GIN -HL variant for OOD data. Ultimately, the configuration with the highest average LF1 score is $h = 2$, $t = 0.5$, and without the hybrid loss (–HL), both for ID and OOD data.

6.3. Effects of different feature embeddings

In this section, we investigate the effects of feature embeddings used on the performance of our model with the highest average LF1 score, where h is 2, t is 0.5, and without the hybrid loss (–HL), and compare it to the two most recent baseline models (GNNSDP and DynGL). Three types of feature embeddings are examined: basic, namely an embedding consisting of only POS tags; +C+L, basic together with character and lemma embeddings; and ++BERT, basic together with character, lemma, and BERT embeddings. The investigation results are presented in Table 5, which shows the average LF1 scores of the HybridGL model along with the standard deviations obtained from five runs. Unfortunately, the standard deviation information for the baseline models was not provided in the previous papers [5,24].

From the table, we can observe that employing more feature embeddings improves performance, sometimes significantly, for the three models and their variants and all datasets. Consistent increases are evident when we compare the basic and +C+L types. In this case, the most significant average accuracy increase is achieved by GNNSDP (GAT), with a 0.8% increase for ID data and 1.2% for OOD data. Similarly, consistent improvements occur when we compare the +C+L and ++BERT types. In this case, the largest average accuracy increase is achieved by DynGL (GCN) with a 1.07% increase for ID data and a 2.3% increase for OOD data. These consistent behaviors suggest that employing the complete feature embeddings (POS, character, lemma, and BERT) is the best option and produces the best accuracies.

Table 4Labeled F1 scores achieved by HybridGL (using HGSL with or without the hybrid loss function) with different numbers of layers h and adjacency matrix thresholds t .

Model	h	t	DM		PAS		PSD		Average	
			ID	OOD	ID	OOD	ID	OOD	ID	OOD
HybridGL (GCN): -HL	2	0.5	96.47 ± 0.07	94.07 ± 0.08	96.43 ± 0.05	94.72 ± 0.15	88.31 ± 0.03	87.81 ± 0.01	93.74 ± 0.05	92.20 ± 0.08
HybridGL (GIN): -HL	2	0.5	96.42 ± 0.04	94.04 ± 0.05	96.48 ± 0.02	94.83 ± 0.07	88.36 ± 0.07	87.92 ± 0.05	93.75 ± 0.04	92.26 ± 0.06
HybridGL (GATv2): -HL	2	0.5	96.44 ± 0.03	94.08 ± 0.05	96.46 ± 0.05	94.85 ± 0.14	88.33 ± 0.08	87.80 ± 0.10	93.74 ± 0.05	92.24 ± 0.10
HybridGL (GCN): -HL	3	0.4	96.43 ± 0.05	93.99 ± 0.05	96.38 ± 0.09	94.78 ± 0.08	88.23 ± 0.04	87.86 ± 0.03	93.68 ± 0.06	92.21 ± 0.05
HybridGL (GIN): -HL	3	0.4	96.54 ± 0.05	94.07 ± 0.06	96.48 ± 0.04	94.83 ± 0.04	88.27 ± 0.08	87.91 ± 0.14	93.77 ± 0.06	92.27 ± 0.08
HybridGL (GATv2): -HL	3	0.4	96.50 ± 0.03	94.10 ± 0.07	96.47 ± 0.04	94.83 ± 0.02	88.02 ± 0.14	87.66 ± 0.14	93.66 ± 0.07	92.19 ± 0.08
HybridGL (GCN): +HL	2	0.5	96.31 ± 0.02	93.96 ± 0.08	96.36 ± 0.02	94.73 ± 0.03	88.46 ± 0.05	88.07 ± 0.14	93.71 ± 0.03	92.25 ± 0.09
HybridGL (GIN): +HL	2	0.5	96.16 ± 0.03	93.73 ± 0.07	96.27 ± 0.07	94.67 ± 0.05	88.24 ± 0.12	87.85 ± 0.09	93.56 ± 0.07	92.08 ± 0.07
HybridGL (GATv2): +HL	2	0.5	96.06 ± 0.18	93.63 ± 0.23	96.28 ± 0.02	94.59 ± 0.05	88.18 ± 0.06	87.82 ± 0.14	93.51 ± 0.08	92.01 ± 0.14
HybridGL (GCN): +HL	3	0.4	96.24 ± 0.04	93.92 ± 0.004	96.32 ± 0.05	94.59 ± 0.02	88.39 ± 0.03	88.02 ± 0.03	93.65 ± 0.04	92.18 ± 0.02
HybridGL (GIN): +HL	3	0.4	96.09 ± 0.05	93.62 ± 0.08	96.34 ± 0.06	94.66 ± 0.03	88.24 ± 0.27	87.76 ± 0.28	93.55 ± 0.13	92.01 ± 0.13
HybridGL (GATv2): +HL	3	0.4	96.02 ± 0.08	93.54 ± 0.12	96.24 ± 0.04	94.55 ± 0.04	88.10 ± 0.17	87.69 ± 0.17	93.45 ± 0.10	91.93 ± 0.11

Table 5

Comparison of labeled F1 scores achieved by HybridGL and the two most recent baseline models with various types of feature embeddings.

Model	DM		PAS		PSD		Average	
	ID	OOD	ID	OOD	ID	OOD	ID	OOD
GNNSDP (GCN): Basic	93.30	88.00	94.80	91.10	85.60	83.60	91.20	87.60
GNNSDP (GAT): Basic	93.00	87.90	94.80	91.60	85.40	83.30	91.10	87.60
HybridGL (GCN): Basic	94.71 ± 0.26	91.32 ± 0.22	95.56 ± 0.03	92.97 ± 0.07	86.61 ± 0.09	85.46 ± 0.21	92.29 ± 0.12	89.92 ± 0.17
HybridGL (GIN): Basic	94.67 ± 0.23	91.43 ± 0.30	95.59 ± 0.03	93.06 ± 0.06	86.62 ± 0.11	85.43 ± 0.17	92.29 ± 0.12	89.97 ± 0.18
HybridGL (GATv2): Basic	94.67 ± 0.26	91.29 ± 0.29	95.61 ± 0.07	93.05 ± 0.05	86.61 ± 0.09	85.42 ± 0.08	92.30 ± 0.14	89.92 ± 0.14
GNNSDP (GCN): +C+L	94.20	90.10	94.90	91.40	86.40	84.90	91.80	88.80
GNNSDP (GAT): +C+L	94.40	89.90	95.00	91.80	86.20	84.60	91.90	88.80
DynGL (GCN): +C+L	95.00	90.10	95.00	92.00	86.60	85.00	92.20	89.00
DynGL (GAT): +C+L	94.90	90.50	95.30	92.10	86.70	85.00	92.30	89.20
HybridGL (GCN): +C+L	95.52 ± 0.24	92.37 ± 0.23	95.72 ± 0.07	93.05 ± 0.12	87.26 ± 0.13	86.08 ± 0.12	92.83 ± 0.15	90.50 ± 0.16
HybridGL (GIN): +C+L	95.56 ± 0.32	92.43 ± 0.37	95.70 ± 0.04	93.08 ± 0.06	87.23 ± 0.12	86.02 ± 0.15	92.83 ± 0.16	90.51 ± 0.19
HybridGL (GATv2): +C+L	95.51 ± 0.25	92.30 ± 0.24	95.66 ± 0.05	93.14 ± 0.07	87.26 ± 0.20	86.23 ± 0.16	92.81 ± 0.17	90.56 ± 0.16
GNNSDP (GCN): ++BERT	95.10	91.10	95.70	93.20	87.70	87.30	92.80	90.50
GNNSDP (GAT): ++BERT	95.30	91.90	96.00	94.30	87.00	86.70	92.80	91.00
DynGL (GCN): ++BERT	95.80	92.70	96.20	94.20	87.80	87.00	93.30	91.30
DynGL (GAT): ++BERT	95.90	92.70	96.20	94.30	87.70	87.20	93.30	91.40
HybridGL (GCN): ++BERT	96.47 ± 0.07	94.07 ± 0.08	96.43 ± 0.05	94.72 ± 0.15	88.31 ± 0.03	87.81 ± 0.01	93.74 ± 0.05	92.20 ± 0.08
HybridGL (GIN): ++BERT	96.42 ± 0.04	94.04 ± 0.05	96.48 ± 0.02	94.83 ± 0.07	88.36 ± 0.07	87.92 ± 0.05	93.75 ± 0.04	92.26 ± 0.06
HybridGL (GATv2): ++BERT	96.44 ± 0.03	94.08 ± 0.05	96.46 ± 0.05	94.85 ± 0.14	88.33 ± 0.08	87.80 ± 0.10	93.74 ± 0.05	92.24 ± 0.10

Furthermore, it is evident that all HybridGL variants constantly improve the accuracy of the two baselines for each type of feature embeddings (basic, +C+L, and ++BERT). For the basic type, HybridGL (GATv2) achieves the best average accuracy for ID with a 1.1% increase, and HybridGL (GIN) for OOD data with a 2.37% increase from the best previous baseline. For the +C+L type, the best average accuracy is achieved by HybridGL (GCN) and HybridGL (GIN) for ID data and HybridGL (GATv2) for OOD data, with 0.93% and 1.36% increases, respectively, from the best previous baseline. Again, we observe consistent behaviors. In this case, the behaviors suggest that all HybridGL variants are inherently better than the two baseline models, independent of the feature embeddings used.

6.4. Main results

In this section, we present the main results by comparing the performance of our proposed model HybridGL, with the best configuration for each variant (marked with *) with all baseline models on the three formalisms of the SemEval-2015 Task 18 English datasets, provided in Table 6. The best HybridGL (GCN) configuration is $h = 2$, $t = 0.5$, and with hybrid loss. The best HybridGL (GIN) configuration is $h = 3$, $t = 0.4$, and without hybrid loss. The best HybridGL (GATv2) configuration is $h = 2$, $t = 0.5$, and without hybrid loss. The details of the baseline models, including the feature embeddings they employ, are specified in Table 3. Table 6 presents the average LF1 scores of the HybridGL model with the standard deviations obtained from five

runs. No standard deviation information for the baseline models was available in the previous papers [5,24].

From Table 6, GNNSDP and DynGL are indeed the previous state-of-the-art methods and perform consistently better than the rest of the baselines. Among them, DynGL (GAT) is almost always the best. More importantly, each variant of HybridGL outperforms all baseline models on all formalisms in the dataset. The average accuracy of the best HybridGL is 0.47% above the best baseline for ID data and 0.87% above the best baseline for OOD data, namely the GIN variant. HybridGL results in a substantial accuracy improvement when applied to the DM formalism, with a 1.38% increase in OOD data using the GATv2 variant. An exciting and subtle property of the proposed model is that each variant reduces the accuracy gap between ID and OOD data, especially for the DM and PAS formalisms.

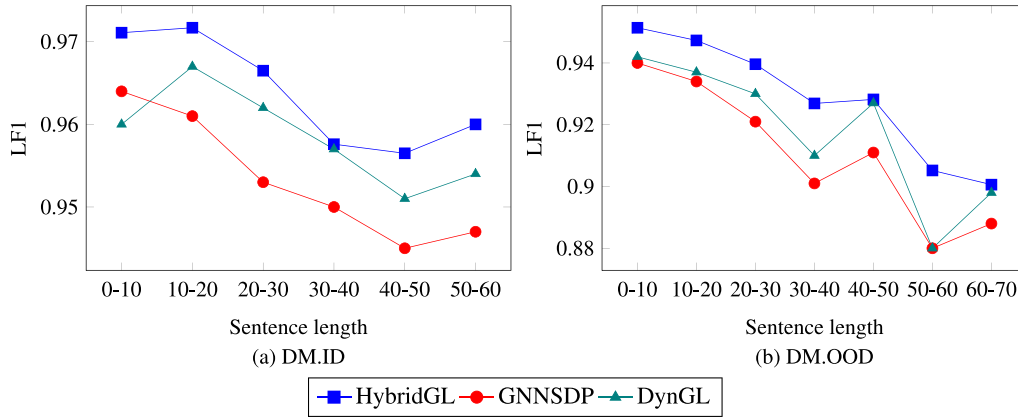
In general, the performance of HybridGL (GIN) and HybridGL (GATv2) is better than that of HybridGL (GCN), with HybridGL (GIN) having the best average accuracy. The substantial performance improvement seen in HybridGL demonstrates that hybrid GSL, an adaptation of DGSLN, can generate enhanced initial static graphs to enhance the accuracy of semantic dependency parsing.

In the following five subsections, we meticulously explore the proposed method. We examine its performance on different sentence lengths and dependency lengths, conduct an ablation study, scrutinize its running speed, and present a case study. This thorough approach ensures the validity and reliability of our findings.

Table 6

Comparison of labeled F1 scores achieved by HybridGL and all baseline models on all formalisms of the English datasets.

Model	DM		PAS		PSD		Average	
	ID	OOD	ID	OOD	ID	OOD	ID	OOD
CompoSem	94.10	90.50	94.70	92.80	82.10	81.60	90.30	88.30
FastSem	93.90	90.40	94.70	92.70	81.90	81.60	90.20	88.20
SemPointer	94.40	91.00	95.10	93.40	82.60	82.00	90.70	88.80
Flair	94.60	90.80	96.10	94.40	86.80	79.50	92.50	88.20
ACE-Fine-tune	95.60	92.60	95.80	94.60	83.80	83.40	91.70	90.20
GNNSDP (GCN)	95.10	91.10	95.70	93.20	87.70	87.30	92.80	90.50
GNNSDP (GAT)	95.30	91.90	96.00	94.30	87.00	86.70	92.80	91.00
DynGL (GCN)	95.80	92.70	96.20	94.20	87.80	87.00	93.30	91.30
DynGL (GAT)	95.90	92.70	96.20	94.30	87.70	87.20	93.30	91.40
HybridGL (GCN)*	96.31 $\pm$ 0.02	93.96 $\pm$ 0.08	96.36 $\pm$ 0.02	94.73 $\pm$ 0.03	88.46 $\pm$ 0.05	88.07 $\pm$ 0.14	93.71 $\pm$ 0.03	92.25 $\pm$ 0.09
HybridGL (GIN)*	96.54 $\pm$ 0.05	94.07 $\pm$ 0.06	96.48 $\pm$ 0.04	94.83 $\pm$ 0.04	88.27 $\pm$ 0.08	87.91 $\pm$ 0.14	93.77 $\pm$ 0.06	92.27 $\pm$ 0.08
HybridGL (GATv2)*	96.44 $\pm$ 0.03	94.08 $\pm$ 0.05	96.46 $\pm$ 0.05	94.85 $\pm$ 0.14	88.33 $\pm$ 0.08	87.80 $\pm$ 0.10	93.74 $\pm$ 0.05	92.24 $\pm$ 0.10

**Fig. 3.** Labeled F1 scores of HybridGL (GIN), GNNSDP (GAT), and DynGL (GAT) for different sentence lengths on the DM formalism.

6.5. Performance on different sentence lengths

By utilizing ID and OOD data from the DM formalism, we analyze our proposed model's performance on various sentence lengths. According to the data ranges, ID data are divided into six groups and OOD data into seven groups, each with a range of ten words. By employing this data partition, we conduct a performance comparison between our top-performing model, HybridGL (GIN), and previously leading models in semantic dependency parsing, specifically GNNSDP (GAT) and DynGL (GAT). The experiment results are depicted in Fig. 3.

The results indicate that HybridGL (GIN) consistently outperforms GNNSDP (GAT) and DynGL (GAT) in all data ranges. Long sentence parsing continues to be a distinctive obstacle in SDP research, in addition to the fact that the performance of every model degrades as the sentence length increases. The results show that the HybridGL (GIN) model can significantly improve accuracy on data with long sentences, especially in DM.OOD data, on the 50–60 words range by 2.86% and on the 30–40 words range by 1.85%. This suggests that HybridGL (GIN) can predict semantic dependency parsing in long sentences more accurately than the best previous models.

6.6. Performance on different dependency lengths

Performance analysis of our best model, HybridGL (GIN), is also carried out for different dependency lengths using ID and OOD data of the DM formalism to then evaluate the results by comparing them to those of the previous best models, GNNSDP (GAT) and DynGL (GAT). The dependency length of a word w_i to another word w_j is simply $|i - j|$. The evaluation is done by dividing the dataset into eight groups and ensuring that in each group, all sentences have at least one dependency with a length according to the group.

Fig. 4 shows the results of the evaluation and comparison. The HybridGL (GIN) model generally has better results compared to the GNNSDP (GAT) and DynGL (GAT) models in dealing with dependencies of different lengths. For short-range dependencies (1–3), the proposed model's accuracy tends to be not significantly different than the previous best models. However, for long-range dependencies (≥ 4), the accuracy of our model is much better. In the DM.OOD dataset with a dependency length of 7, our model can improve accuracy by 3.97%, and with a dependency length of ≥ 8 , our model can improve accuracy by 4.16%. The findings demonstrate that our model outperforms the previous model in capturing long-range dependencies.

6.7. Ablation study

In this ablation study, we investigate how the number of hybrid GSL layers and the number of layers on the GNN variants affect our parser model. Training and testing on our best model, namely HybridGL (GIN), is done on the DM formalism using ++BERT embedding settings. The number of hybrid GSL layers is increased from 1 to 3. Besides, we reduce the number of layers in GIN from 3 to 1 by choosing the number of hybrid GSL layers with the best test results. The experiment results are presented in Table 7, which shows the average LF1 scores of the HybridGL model along with the standard deviations obtained from five runs. Unfortunately, the standard deviation information for the baseline models (DynGL (GAT)) was not provided in the previous papers [5,24].

The results indicate that the HybridGL model performs better for all the tested combinations of the number of hybrid GSL layers and GIN layers than DynGL (GAT). This suggests that using a hybrid GSL model to obtain an adjacency matrix is beneficial for improving the performance of the parser model. Besides, the use of a GNN variant, namely GIN, also impacts the parser's performance. As the number

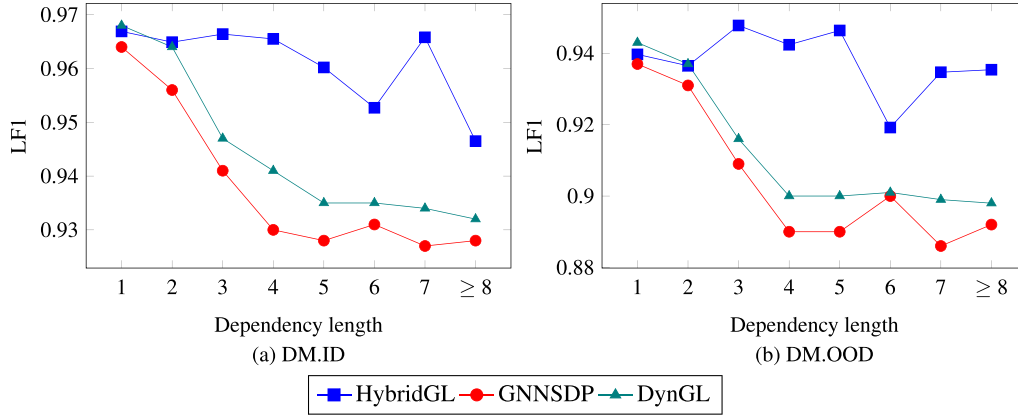


Fig. 4. Labeled F1 scores of HybridGL (GIN), GNNSDP (GAT), and DynGL (GAT) for different dependency lengths on the DM formalism.

Table 7

Labeled F1 scores achieved by our model with different numbers of hybrid GSL and GNN layers.

Model	GSL layers	GNN layers	DM	
			ID	OOD
DynGL (GAT)	MLP	3	95.90	92.70
HybridGL (GIN)	hybrid GSL: 1	3	96.53 ± 0.03	94.05 ± 0.04
HybridGL (GIN)	hybrid GSL: 2	3	96.42 ± 0.04	94.04 ± 0.05
HybridGL (GIN)	hybrid GSL: 3	3	96.54 ± 0.05	94.07 ± 0.06
HybridGL (GIN)	hybrid GSL: 3	2	96.45 ± 0.004	94.07 ± 0.05
HybridGL (GIN)	hybrid GSL: 3	1	96.47 ± 0.04	94.06 ± 0.05

Table 8

The models' parsing speed (sentences/second).

Model	Parsing speed
DynGL (GAT)	474
HybridGL (GCN)	288
HybridGL (GIN)	385
HybridGL (GATv2)	207

of hybrid GSL and GNN layers increases, so does the performance of HybridGL. In ID data, the best performance is obtained when the number of hybrid GSL and GNN layers is 3, while in OOD data, when the number of hybrid GSL layers is 3 and the number of GNN layers is 2 or 3.

6.8. Running times

In addition to accuracy, parsing speed can determine whether a parser can be applied to downstream tasks. Therefore, we compare the parsing speed of DynGL (GAT) with our three best models: HybridGL (GCN), HybridGL (GIN), and HybridGL (GATv2), on the semantic dependency parsing of 1,849 OOD data from the DM formalism. The parsing process is run on a machine with NVIDIA GeForce RTX 4060 GPUs. The results are shown in Table 8.

The results show that DynGL is faster in parsing than all variants of HybridGL, although the HybridGL models have better accuracy. Among the HybridGL variants, the GIN variant is the fastest. This demonstrates that although the hybrid GSL has been shown to improve the performance of the parsing model, it compensates for a slower parsing process than the DynGL model.

6.9. Case study

In this brief case study, we demonstrate how HybridGL (GIN) can do better than DynGL by using the hybrid GSL adjacency matrix and GIN as the GNN variant in the learning process of the graph representation. We select a sentence with *sent_id* = 40510098 from OOD data of the DM

formalism: “They are often ignorant as well as underprivileged”. Figs. 5(a) and 5(b) depict the adjacency matrices that need to be added to the learning process of the graph representation on DynGL and HybridGL models, respectively. Figs. 5(c) and 5(d) show the semantic dependency graphs due to parsing on DynGL and HybridGL models, respectively. Both parser models are implemented with the ++BERT embedding settings.

From Fig. 5(a), we can see two errors: (1) An error in detecting a dependency that should not exist, i.e., dependency “well-underprivileged”; and (2) A dependency that cannot be detected but should be there, i.e., dependency “underprivileged - They”. These two mistakes indicate that the input to the adjacency matrix when MLP is used as GSL in the DynGL model is still noisy. Hence, semantic dependency graphs obtained from incorrect adjacency matrices produce erroneous parsing results. Consequently, we can see in Fig. 5(c) that the semantic dependency graph of the DynGL parsing result has 3 errors: (1) A dependency detection error and a wrong dependency label on dependency “well - underprivileged” that should not exist; (2) An error in detecting the dependency label of “ignorant - underprivileged” which should have the correct label “_as+well+as_c”; and (3) An undetected dependency and the dependency label of “underprivileged - They”.

Unlike the DynGL parsing result, HybridGL’s adjacency matrix result, which uses the hybrid GSL, generates the correct graph structure. Additionally, HybridGL’s final semantic dependency graph, which uses GIN in the graph representation learning step, also produces the correct results. Furthermore, this demonstrates that our model, which adheres to the three fundamental principles of graph generation through the use of hybrid GSL, is more effective in preventing errors and delivering more precise results.

7. Conclusion

This paper introduces HybridGL, a semantic dependency parsing model. The HybridGL model employs the same framework as DynGL but uses the hybrid GSL to construct the adjacency matrix. In contrast, the DynGL model uses an MLP and a biaffine attention network at GSL. This method uses solely feature similarity (homophily). Our hybrid

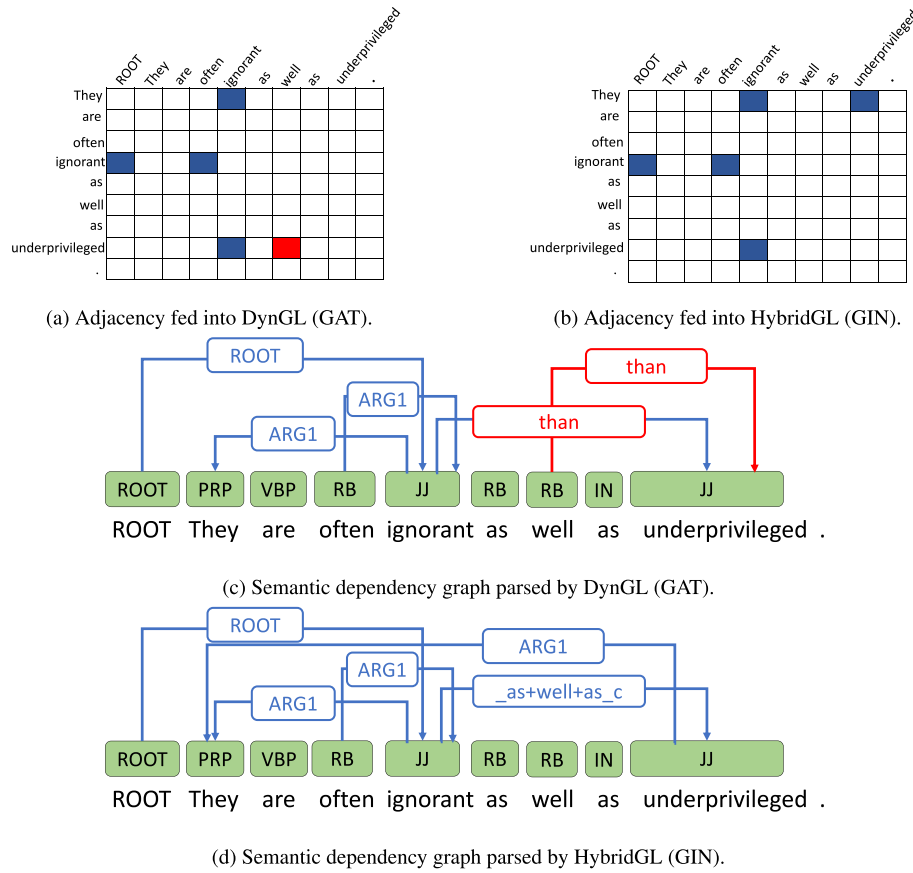


Fig. 5. Adjacency matrices generated by the GSLs and passed through two parsers, DynGL and HybridGL, respectively (a)–(b), and the semantic dependency graphs as the parsing results of the two parsers (c)–(d).

GSL is a modified DGSLN approach that meets homophily, sparsity, and degree distribution. Our model differs from DGSLN at the final stage, where it employs a threshold to build the adjacency matrix. Additionally, our model uses GIN and GATv2 as GNN learning variants for the graph representation stage, while DynGL uses GCN and GAT.

Based on the experiments on the English SemEval 2015 Task 18 dataset, increasing the GNN variant layers enhances model accuracy. However, adding HGSL layers generally does not enhance accuracy. In DGSLN, hybrid loss functions increase graph and node classification accuracy. However, our semantic dependency parsing study only found that the hybrid loss function enhances accuracy with the GCN variant and PSD formalism. In testing with varied numbers of embeddings, all HybridGL variations exceed the accuracy of GNNSDP and DynGL for each type of feature embeddings (basic, +C+L, and ++BERT). Our model outperforms all baseline models with an average F1 score of 93.77% on in-domain datasets with the GIN variant and 92.27% on out-of-domain datasets, also with the GIN variant. Besides the analytical results, HybridGL parses long sentences and long-range dependencies better than DynGL and GNNSDP.

Despite significant accuracy increases, further research is needed to overcome current limitations. HybridGL models are more accurate than DynGL but slower at parsing. Enhancing the hybrid GSL technique to maximize efficiency and speed could fix this. The hybrid loss function can be improved in the future by using regularization techniques like contrastive learning-based regularization and heterophily-aware graph propagation, which better accommodate datasets such as SDP that tend to be heterophily. In addition, using the Transformer in contextual representation learning and other BERT variant models like RoBERTa or SpanBERT for feature embedding may improve accuracy. Finally, a graph self-supervised learning model is needed to overcome the enormous labeled data requirement of the training process.

CRediT authorship contribution statement

Irkham Huda: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Moh Edi Wibowo:** Writing – review & editing, Validation, Supervision, Formal analysis. **Reza Pulungan:** Writing – review & editing, Validation, Supervision, Methodology, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Irkham Huda reports financial support was provided by the Ministry of Higher Education, Science, and Technology of the Republic of Indonesia. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Irkham Huda thanks the Center for Higher Education Funding and Assessment (PPAPT) of the Ministry of Higher Education, Science, and Technology of the Republic of Indonesia and the Indonesia Endowment Fund for Education (LPDP) for funding his doctoral study through the Indonesian Education Scholarship (BPI).

Data availability

We have created a repository to store the source code and data in <https://simpan.ugm.ac.id/s/CBRvB2mNi1JdsQ1>.

References

- [1] J. Li, M. Zhu, W. Lu, G. Zhou, Improving semantic parsing with enriched synchronous context-free grammars in statistical machine translation, *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* 16 (1) (2016) <http://dx.doi.org/10.1145/2963099>.
- [2] H. Li, E. Hamzei, I. Majic, H. Hua, J. Renz, M. Tomko, M. Vasardani, S. Winter, T. Baldwin, Neural factoid geospatial question answering, *J. Spat. Inf. Sci.* (23) (2021) 65–90, <http://dx.doi.org/10.5311/josis.2021.23.159>.
- [3] H. Zou, X. Tang, B. Xie, B. Liu, Sentiment classification using machine learning techniques with syntax features, in: 2015 International Conference on Computational Science and Computational Intelligence, CSCI, 2015, pp. 175–179, <http://dx.doi.org/10.1109/CSCI.2015.44>.
- [4] U. Taufiq, R. Pulungan, Y. Suyanto, Named entity recognition and dependency parsing for better concept extraction in summary obfuscation detection, *Expert Syst. Appl.* 217 (2023) 119579, <http://dx.doi.org/10.1016/j.eswa.2023.119579>.
- [5] B. Li, Y. Fan, Y. Sataer, Z. Gao, Y. Gui, Improving semantic dependency parsing with higher-order information encoded by graph neural networks, *Appl. Sci.* 12 (8) (2022) <http://dx.doi.org/10.3390/app12084089>.
- [6] M. Lindemann, J. Groschwitz, A. Koller, Compositional semantic parsing across graphbanks, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Florence, Italy, 2019, pp. 4576–4585, <http://dx.doi.org/10.18653/v1/P19-1450>.
- [7] D. Fernández-González, C. Gómez-Rodríguez, Transition-based semantic dependency parsing with pointer networks, in: D. Jurafsky, J. Chi, N. Schluter, J.R. Tetraault (Eds.), Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5–10, 2020, Association for Computational Linguistics, 2020, pp. 7035–7046, <http://dx.doi.org/10.18653/v1/2020.acl-main.629>.
- [8] J. Nivre, Algorithms for deterministic incremental dependency parsing, *Comput. Linguist.* 34 (4) (2008) 513–553, <http://dx.doi.org/10.1162/coli.07-056-R1-07-027>.
- [9] Y. Zhang, S. Clark, A tale of two parsers: Investigating and combining graph-based and transition-based dependency parsing using beam-search, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing, EMNLP '08, Association for Computational Linguistics, USA, 2008, pp. 562–571.
- [10] Y. Zhang, J. Nivre, Transition-based dependency parsing with rich non-local features, in: Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies: Short Papers - Volume 2, HLT '11, Association for Computational Linguistics, USA, 2011, pp. 188–193.
- [11] Y. Goldberg, J. Nivre, A dynamic oracle for arc-eager dependency parsing, in: M. Kay, C. Boitet (Eds.), COLING 2012, 24th International Conference on Computational Linguistics, Proceedings of the Conference: Technical Papers, 8–15 December 2012, Mumbai, India, Indian Institute of Technology Bombay, 2012, pp. 959–976.
- [12] D. Chen, C.D. Manning, A fast and accurate dependency parser using neural networks, in: A. Moschitti, B. Pang, W. Daelemans (Eds.), Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25–29, 2014, Doha, Qatar, ACL, 2014, pp. 740–750, <http://dx.doi.org/10.3115/v1/d14-1082>.
- [13] M. Ballesteros, C. Dyer, Y. Goldberg, N.A. Smith, Greedy transition-based dependency parsing with stack LSTMs, *Comput. Linguist.* 43 (2) (2017) 311–347, http://dx.doi.org/10.1162/COLI_a.00285.
- [14] X. Ma, Z. Hu, J. Liu, N. Peng, G. Neubig, E.H. Hovy, Stack-pointer networks for dependency parsing, in: I. Gurevych, Y. Miyao (Eds.), Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018, Melbourne, Australia, July 15–20, 2018, Volume 1: Long Papers, Association for Computational Linguistics, 2018, pp. 1403–1414, <http://dx.doi.org/10.18653/v1/P18-1130>.
- [15] M. Zhang, A survey of syntactic-semantic parsing based on constituent and dependency structures, *Sci. China Technol. Sci.* 63 (10) (2020) 1898–1920, <http://dx.doi.org/10.1007/s11431-020-1666-4>.
- [16] W. Chen, J. Kazama, K. Uchimoto, K. Torisawa, Improving dependency parsing with subtrees from auto-parsed data, in: Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing: Volume 2 - Volume 2, EMNLP '09, Association for Computational Linguistics, USA, 2009, pp. 570–579.
- [17] R.T. McDonald, K. Crammer, F.C.N. Pereira, Online large-margin training of dependency parsers, in: K. Knight, H.T. Ng, K. Oflazer (Eds.), ACL 2005, 43rd Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, 25–30 June 2005, University of Michigan, USA, The Association for Computational Linguistics, 2005, pp. 91–98, <http://dx.doi.org/10.3115/1219840.1219852>.
- [18] T. Koo, M. Collins, Efficient third-order dependency parsers, in: J. Hajič, S. Carberry, S. Clark (Eds.), ACL 2010, Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, July 11–16, 2010, Uppsala, Sweden, The Association for Computational Linguistics, 2010, pp. 1–11.
- [19] X. Ma, H. Zhao, Fourth-order dependency parsing, in: M. Kay, C. Boitet (Eds.), COLING 2012, 24th International Conference on Computational Linguistics, Proceedings of the Conference: Posters, 8–15 December 2012, Mumbai, India, Indian Institute of Technology Bombay, 2012, pp. 785–796.
- [20] Z. Zhang, H. Zhao, L. Qin, Probabilistic graph-based dependency parsing with convolutional neural network, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7–12, 2016, Berlin, Germany, Volume 1: Long Papers, The Association for Computational Linguistics, 2016, <http://dx.doi.org/10.18653/v1/p16-1131>.
- [21] E. Kiperwasser, Y. Goldberg, Simple and accurate dependency parsing using bidirectional LSTM feature representations, *Trans. Assoc. Comput. Linguist.* 4 (2016) 313–327, http://dx.doi.org/10.1162/tacl_a.00101.
- [22] T. Dozat, C.D. Manning, Deep biaffine attention for neural dependency parsing, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, OpenReview.net, 2017.
- [23] X. Wang, J. Huang, K. Tu, Second-order semantic dependency parsing with end-to-end neural networks, in: A. Korhonen, D.R. Traum, L. Màrquez (Eds.), Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28–August 2, 2019, Volume 1: Long Papers, Association for Computational Linguistics, 2019, pp. 4609–4618, <http://dx.doi.org/10.18653/v1/p19-1454>.
- [24] B. Li, Y. Fan, M. Gao, Y. Sataer, Z. Gao, A joint-learning-based dynamic graph learning framework for structured prediction, *Electronics* 12 (11) (2023) <http://dx.doi.org/10.3390/electronics12112357>.
- [25] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, OpenReview.net, 2017.
- [26] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30–May 3, 2018, OpenReview.net, 2018.
- [27] S. Brody, U. Alon, E. Yahav, How attentive are graph attention networks? in: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022, OpenReview.net, 2022.
- [28] W.L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS '17, Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 1025–1035.
- [29] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks? in: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019, OpenReview.net, 2019.
- [30] X. Zou, K. Li, C. Chen, X. Yang, W. Wei, K. Li, DGLSLN: Differentiable graph structure learning neural network for robust graph representations, *Inform. Sci.* 626 (2023) 94–113, <http://dx.doi.org/10.1016/j.ins.2023.01.059>.
- [31] Z. Zhang, J. Bu, M. Ester, J. Zhang, C. Yao, Z. Yu, C. Wang, Hierarchical graph pooling with structure learning, 2019, *CoRR arXiv:1911.05954*.
- [32] S. Luan, C. Hua, M. Xu, Q. Lu, J. Zhu, X. Chang, J. Fu, J. Leskovec, D. Precup, When do graph neural networks help with node classification? Investigating the homophily principle on node distinguishability, in: A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (Eds.), Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023, 2023.
- [33] J. Hajič, M. Ciaramita, R. Johansson, D. Kawahara, M.A. Martí, L. Màrquez, A. Meyers, J. Nivre, S. Padó, J. Štěpánek, P. Straňák, M. Surdeanu, N. Xue, Y. Zhang, The CoNLL-2009 shared task: Syntactic and semantic dependencies in multiple languages, in: Proceedings of the Thirteenth Conference on Computational Natural Language Learning: Shared Task, CoNLL '09, Association for Computational Linguistics, USA, 2009, pp. 1–18.
- [34] S. Oepen, M. Kuhlmann, Y. Miyao, D. Zeman, S. Cinková, D. Flickinger, J. Hajič, Z. Uresová, SemEval 2015 Task 18: Broad-coverage semantic dependency parsing, in: D.M. Cer, D. Jurgens, P. Nakov, T. Zesch (Eds.), Proceedings of the 9th International Workshop on Semantic Evaluation, SemEval@NAACL-HLT 2015, Denver, Colorado, USA, June 4–5, 2015, The Association for Computer Linguistics, 2015, pp. 915–926, <http://dx.doi.org/10.18653/V1/S15-2153>.
- [35] J. Nivre, An efficient algorithm for projective dependency parsing, in: Proceedings of the Eighth International Conference on Parsing Technologies, IWPT 2003, Nancy, France, April 2003, 2003, pp. 149–160.
- [36] C. Gómez-Rodríguez, J. Nivre, Divisible transition systems and multiplanar dependency parsing, *Comput. Linguist.* 39 (4) (2013) 799–845, http://dx.doi.org/10.1162/COLI_A.00150.
- [37] Y. Yuan, Y. Jiang, K. Tu, Bidirectional transition-based dependency parsing, *Proc. AAAI Conf. Artif. Intell.* 33 (01) (2019) 7434–7441, <http://dx.doi.org/10.1609/aaai.v33i01.33017434>.
- [38] D. Fernández-González, C. Gómez-Rodríguez, Dependency parsing with bottom-up hierarchical pointer networks, *Inf. Fusion* 91 (2023) 494–503, <http://dx.doi.org/10.1016/j.inffus.2022.10.023>.
- [39] R.T. McDonald, F.C.N. Pereira, Online learning of approximate dependency parsing algorithms, in: D. McCarthy, S. Wintner (Eds.), EACL 2006, 11th Conference of the European Chapter of the Association for Computational Linguistics, Proceedings of the Conference, April 3–7, 2006, Trento, Italy, The Association for Computational Linguistics, 2006.

- [40] T. Ji, Y. Wu, M. Lan, Graph-based dependency parsing with graph neural networks, in: A. Korhonen, D.R. Traum, L. Màrquez (Eds.), Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28–August 2, 2019, Volume 1: Long Papers, Association for Computational Linguistics, 2019, pp. 2475–2485, <http://dx.doi.org/10.18653/V1/P19-1237>.
- [41] T. Dozat, C.D. Manning, Simpler but more accurate semantic dependency parsing, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), Association for Computational Linguistics, Melbourne, Australia, 2018, pp. 484–490, <http://dx.doi.org/10.18653/v1/P18-2077>.
- [42] R. Li, S. Wang, F. Zhu, J. Huang, Adaptive graph convolutional neural networks, Proc. AAAI Conf. Artif. Intell. 32 (1) (2018) <http://dx.doi.org/10.1609/aaai.v32i1.11691>.
- [43] J. Zhao, X. Wang, C. Shi, B. Hu, G. Song, Y. Ye, Heterogeneous graph structure learning for graph neural networks, Proc. AAAI Conf. Artif. Intell. 35 (5) (2021) 4697–4705, <http://dx.doi.org/10.1609/aaai.v35i5.16600>.
- [44] J. Yuan, M. Cao, H. Cheng, H. Yu, J. Xie, C. Wang, A unified structure learning framework for graph attention networks, Neurocomputing 495 (2022) 194–204, <http://dx.doi.org/10.1016/j.neucom.2022.01.064>.
- [45] D.S. Pilco, A.R. Rivera, Graph learning network: A structure learning algorithm, 2019, CoRR [arXiv:1905.12665](https://arxiv.org/abs/1905.12665).
- [46] B. Jiang, Z. Zhang, D. Lin, J. Tang, B. Luo, Semi-supervised learning with graph learning-convolutional networks, in: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2019, pp. 11305–11312, <http://dx.doi.org/10.1109/CVPR.2019.01157>.
- [47] Y. Zhu, W. Xu, J. Zhang, Q. Liu, S. Wu, L. Wang, Deep graph structure learning for robust representations: A survey, 2021, CoRR [arXiv:2103.03036](https://arxiv.org/abs/2103.03036).
- [48] W. Jin, Y. Ma, X. Liu, X. Tang, S. Wang, J. Tang, Graph structure learning for robust graph neural networks, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 66–74, <http://dx.doi.org/10.1145/3394486.3403049>.
- [49] Z. Duan, H. Xu, Y. Huang, J. Feng, Y. Wang, Multivariate time series forecasting with transfer entropy graph, Tsinghua Sci. Technol. 28 (1) (2023) 141–149, <http://dx.doi.org/10.26599/TST.2021.9010081>.
- [50] Z. Duan, H. Xu, Y. Wang, Y. Huang, A. Ren, Z. Xu, Y. Sun, W. Wang, Multivariate time-series classification with hierarchical variational graph pooling, Neural Netw. 154 (2022) 481–490, <http://dx.doi.org/10.1016/j.neunet.2022.07.032>.
- [51] D. An, Z. Pan, Q. Zhao, W. Liu, J. Liu, Unsupervised graph structure learning based on optimal graph topology modeling and adaptive data augmentation, Mathematics 12 (13) (2024) <http://dx.doi.org/10.3390/math12131991>.
- [52] A. Sperduti, A. Starita, Supervised neural networks for the classification of data structures, IEEE Trans. Neural Netw. 8 (3) (1997) 714–735, <http://dx.doi.org/10.1109/72.572108>.
- [53] P. Frasconi, M. Gori, A. Sperduti, A general framework for adaptive processing of data structures, IEEE Trans. Neural Netw. 9 (5) (1998) 768–786, <http://dx.doi.org/10.1109/72.712151>.
- [54] F. Scarselli, M. Gori, A.C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, IEEE Trans. Neural Netw. 20 (1) (2009) 61–80, <http://dx.doi.org/10.1109/TNN.2008.2005605>.
- [55] A. Micheli, Neural network for graphs: A contextual constructive approach, IEEE Trans. Neural Netw. 20 (3) (2009) 498–511, <http://dx.doi.org/10.1109/TNN.2008.2010350>.
- [56] Y. Lecun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE 86 (11) (1998) 2278–2324, <http://dx.doi.org/10.1109/5.726791>.
- [57] C. Merkwirth, T. Lengauer, Automatic generation of complementary descriptors with molecular graph networks, J. Chem. Inf. Model. 45 (5) (2005) 1159–1168, PMID: 16180893, <http://dx.doi.org/10.1021/ci049613b>.
- [58] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, M. Sun, Graph neural networks: A review of methods and applications, AI Open 1 (2020) 57–81, <http://dx.doi.org/10.1016/j.aiopen.2021.01.001>.
- [59] J. Atwood, D. Towsley, Diffusion-convolutional neural networks, in: Proceedings of the 30th International Conference on Neural Information Processing Systems, NIPS '16, Curran Associates Inc., Red Hook, NY, USA, 2016, pp. 2001–2009.
- [60] A.L. Maas, A.Y. Hannun, A.Y. Ng, Rectifier nonlinearities improve neural network acoustic models, in: Proceedings of the 30th International Conference on Machine Learning, vol. 28, Atlanta, GA, 2013, pp. 1–6.
- [61] M. Khalid, J. Baber, M.K. Kasi, M. Bakhtyar, V. Devi, N. Sheikh, Empirical evaluation of activation functions in deep convolution neural network for facial expression recognition, in: 2020 43rd International Conference on Telecommunications and Signal Processing, TSP, 2020, pp. 204–207, <http://dx.doi.org/10.1109/TSP49548.2020.9163446>.
- [62] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: Y. Bengio, Y. LeCun (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, 2015.
- [63] J. Pennington, R. Socher, C.D. Manning, Glove: Global vectors for word representation, in: A. Moschitti, B. Pang, W. Daelemans (Eds.), Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25–29, 2014, Doha, Qatar, a Meeting of SIGDAT, a Special Interest Group of the ACL, ACL, 2014, pp. 1532–1543, <http://dx.doi.org/10.3115/V1/D14-1162>.
- [64] D. Flickinger, Y. Zhang, V. Kordoni, DeepBank: A dynamically annotated treebank of the Wall Street Journal, in: Proceedings of the 11th International Workshop on Treebanks and Linguistic Theories, 2012, pp. 85–96.
- [65] Y. Miyao, J. Tsujii, Deep linguistic analysis for the accurate identification of predicate-argument relations, in: Proceedings of the 20th International Conference on Computational Linguistics, COLING '04, Association for Computational Linguistics, USA, 2004, pp. 1392–es, <http://dx.doi.org/10.3115/1220355.1220559>.
- [66] J. Hajič, E. Hajičová, J. Panevová, P. Sgall, O. Bojar, S. Cinková, E. Fucíková, M. Mikulová, P. Pajas, J. Popelka, J. Semecký, J. Sindlerová, J. Stepánek, J. Toman, Z. Uresová, Z. Zabokrtský, Announcing Prague Czech-english dependency treebank 2.0, in: N. Calzolari, K. Choukri, T. Declerck, M.U. Dogan, B. Maegaard, J. Mariani, J. Odijk, S. Piperidis (Eds.), Proceedings of the Eighth International Conference on Language Resources and Evaluation, LREC 2012, Istanbul, Turkey, May 23–25, 2012, European Language Resources Association (ELRA), 2012, pp. 3153–3160.
- [67] M. Lindemann, J. Groschwitz, A. Koller, Fast semantic parsing with well-typedness guarantees, in: B. Webber, T. Cohn, Y. He, Y. Liu (Eds.), Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16–20, 2020, Association for Computational Linguistics, 2020, pp. 3929–3951, <http://dx.doi.org/10.18653/V1/2020.EMNLP-MAIN.323>.
- [68] H. He, J.D. Choi, Establishing strong baselines for the new decade: Sequence tagging, syntactic and semantic parsing with BERT, in: R. Barták, E. Bell (Eds.), Proceedings of the Thirty-Third International Florida Artificial Intelligence Research Society Conference, Originally To Be Held in North Miami Beach, Florida, USA, May 17–20, 2020, AAAI Press, 2020, pp. 228–233.
- [69] X. Wang, Y. Jiang, N. Bach, T. Wang, Z. Huang, F. Huang, K. Tu, Automated concatenation of embeddings for structured prediction, in: C. Zong, F. Xia, W. Li, R. Navigli (Eds.), Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1–6, 2021, Association for Computational Linguistics, 2021, pp. 2643–2660, <http://dx.doi.org/10.18653/V1/2021.ACL-LONG.206>.
- [70] O. Platonov, D. Kuznedelev, M. Diskin, A. Babenko, L. Prokhorenkova, A critical look at the evaluation of GNNs under heterophily: Are we really making progress? in: The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1–5, 2023, OpenReview.net, 2023.



Irkham Huda earned his bachelor's and master's degrees in Computer Science in 2012 and 2015, both from the Department of Computer Science and Electronics, Universitas Gadjah Mada. He is currently pursuing his doctoral degree in Computer Science at the Department of Computer Science and Electronics, Universitas Gadjah Mada. He is also a lecturer at the School of Vocational Studies, Universitas Gadjah Mada. His research interests include natural language processing, machine learning, geographic information systems, and mobile engineering.



Moh Edi Wibowo, Ph.D., received his bachelor's degree and master's degree in Computer Science from Universitas Gadjah Mada in 2004 and 2006, respectively, and the Ph.D. degree in Computer Science from Queensland University of Technology in 2014. His current focus area is human and object detection and identification. He is currently a lead researcher at the Laboratory of Algorithms and Computation, Department of Computer Science and Electronics, Universitas Gadjah Mada.



Reza Pulungan, Dr.-Ing., is a professor at the Department of Computer Science and Electronics, Universitas Gadjah Mada. His research background is in stochastic processes, especially Markov processes and phase-type distributions, and modeling and analysis of reactive systems. Currently, he is interested in and working on learning algorithms, including reinforcement learning, and their applications in diverse fields.